MASTER THESIS — CLAUDE CODE BUILD PLAN

Causal Explainability for AI-Driven Investment Decisions

Frankfurt School of Finance & Management | 2026

PASTE THIS ENTIRE DOCUMENT INTO CLAUDE CODE AS YOUR BUILD PROMPT

PROJECT OVERVIEW

Build a complete, reproducible, thesis-grade causal explainability pipeline for AI-driven investment decisions using real market data (DAX + S&P 500). The pipeline must classify each stock as **Buy** / **Hold** / **Sell** using CatBoost, apply DirectLiNGAM causal discovery, compute LEWIS counterfactual scores, compare against SHAP baseline, and produce all evaluation artifacts required for a master thesis submission.

Language: Python
Data: Real-time via Yahoo Finance API (yfinance) — NO synthetic data generation
Target: ~520+ companies (DAX 40 + S&P 500)
Label: Buy / Hold / Sell derived from real forward 3-month returns

---

DIRECTORY STRUCTURE TO CREATE

```
thesis_pipeline/
├── data/
│   ├── raw/                    # Raw downloaded price + macro data
│   └── processed/              # Cleaned, engineered, labeled dataset
├── src/
│   ├── 01_data_download.py     # Step 1: Download real data
│   ├── 02_feature_engineering.py  # Step 2: Compute features
│   ├── 03_labeling.py          # Step 3: Generate Buy/Hold/Sell labels
│   ├── 04_data_preparation.py  # Step 4: Preprocessing, stationarity, split
│   ├── 05_baseline_model.py    # Step 5: CatBoost + SHAP baseline
│   ├── 06_causal_discovery.py  # Step 6: DirectLiNGAM + PC cross-check
│   ├── 07_lewis_scores.py      # Step 7: LEWIS Nec/Suf/Nesuf scores
│   ├── 08_evaluation.py        # Step 8: Full evaluation framework
│   ├── 09_comparison.py        # Step 9: SHAP vs LEWIS comparison
│   ├── 10_governance.py        # Step 10: Regulatory mapping output
│   └── pipeline.py             # Master runner: runs all steps in order
├── models/                     # Saved CatBoost models + adjacency matrices
```

```
├── results/
│   ├── plots/                  # All charts and visualizations
│   ├── tables/                 # CSV exports of all metrics
│   └── reports/                # Summary text outputs
├── notebooks/
│   └── thesis_exploration.ipynb   # Exploratory notebook (optional)
├── requirements.txt
└── README.md
```

STEP-BY-STEP BUILD INSTRUCTIONS

STEP 1 — DATA DOWNLOAD (`src/01_data_download.py`)

Download real price and macro data. No synthetic generation at any point.

Tickers to download:

- DAX 40 companies (full list: ADS.DE, AIR.DE, ALV.DE, BAS.DE, BAYN.DE, BEI.DE, BMW.DE, BNR.DE, CON.DE, 1COV.DE, DB1.DE, DBK.DE, DHL.DE, DTE.DE, EOAN.DE, FRE.DE, HEI.DE, HEN3.DE, IFX.DE, LIN.DE, MBG.DE, MRK.DE, MTX.DE, MUV2.DE, P911.DE, PAH3.DE, QGEN.DE, RWE.DE, SAP.DE, SHL.DE, SIE.DE, SRT.DE, SY1.DE, VNA.DE, VOW3.DE, ZAL.DE)
- S&P 500 companies (use pandas_datareader or yfinance bulk download)

Price data:

- Period: 2015-01-01 to present (minimum 8 years for regime diversity)
- Frequency: Daily OHLCV

Macro features to download from FRED / ECB SDW / Yahoo Finance:

- VIX: ^VIX (Yahoo Finance)
- ECB rate: ECBDFR (FRED)
- EUR/USD: EURUSD=X (Yahoo Finance)
- US 10Y yield: ^TNX (Yahoo Finance)
- DE 10Y yield: ^IRDE10Y or via ECB SDW
- German CPI: DEUCPIALLMINMEI (FRED)
- US CPI: CPIAUCSL (FRED)
- S&P 500 index: ^GSPC
- DAX index: ^GDAXI

Save outputs:

- `data/raw/prices_{ticker}.csv` for each ticker
- `data/raw/macro_data.csv` for macro features
- Log download success/failure count

STEP 2 — FEATURE ENGINEERING (`src/02_feature_engineering.py`)

Compute the following features per ticker per date. All features must be computed from REAL price data only — no imputation with synthetic values.

Stock-level features:

```
volatility      = 20-day rolling std of log returns (annualized)
momentum_1m     = 1-month return (21 trading days)
momentum_3m     = 3-month return (63 trading days)
momentum_6m     = 6-month return (126 trading days)
return_1y       = 1-year return (252 trading days)
max_drawdown    = max peak-to-trough drop in trailing 252 days
volume_avg      = 20-day average volume (log-scaled)
rsi_14          = 14-day Relative Strength Index
macd_signal     = MACD signal line (12/26/9 EMA)
beta_market     = 60-day rolling beta against index (DAX for DE, S&P500 for US)
pe_ratio        = if available from yfinance .info
```

Macro features (aligned to daily trading dates):

```
ecb_rate        = ECB deposit facility rate (forward-filled)
us_10y_yield    = US 10-year Treasury yield
vix             = CBOE VIX index
eur_usd         = EUR/USD exchange rate
de_inflation    = German CPI YoY %
us_inflation    = US CPI YoY %
```

CRITICAL — Stationarity handling:

- Run Augmented Dickey-Fuller (ADF) test on each feature
- For non-stationary features: apply first-difference or log-difference
- Document which features required transformation
- Save stationarity test results to `results/tables/stationarity_report.csv`

STEP 3 — LABELING (`src/03_labeling.py`)

Generate Buy/Hold/Sell labels from REAL forward returns. **NO rule-based or synthetic labeling.**

Method:

```python
forward_return_3m = (price_t+63 / price_t) - 1

# Sector-adjusted thresholds (compute per GICS sector):
sector_median = median forward return per sector per quarter
sector_std    = std of forward return per sector per quarter

buy_threshold  = sector_median + 0.5 * sector_std
sell_threshold = sector_median - 0.5 * sector_std

label = "Buy"  if forward_return_3m > buy_threshold
label = "Sell" if forward_return_3m < sell_threshold
label = "Hold" otherwise
```

Justify thresholds in code comments:

- Sector-adjusted avoids labeling all tech as Buy in bull market
- 0.5 std gives roughly balanced classes

Check class balance:

- Print class distribution per sector, per year
- If severely imbalanced (>70/15/15), document it — do NOT resample artificially
- Save label distribution to `results/tables/label_distribution.csv`

STEP 4 — DATA PREPARATION (`src/04_data_preparation.py`)

Follow the ML course structure exactly. This is where all preprocessing happens.

4.1 — Missing Value Handling:

```python
# Check NA counts per feature
# Forward-fill macro features (they change infrequently)
# Drop rows where stock-level features are NA (don't impute prices)
# Log: rows dropped vs total rows
```

4.2 — Outlier Handling:

```python
# Winsorize at 1st and 99th percentile per feature
# Document which features had significant outliers
```

4.3 — Feature Scaling:

```python
# Use StandardScaler (z-score) for continuous features
# CRITICAL: Fit scaler ONLY on training data, apply to test/validation
# Save scaler object for inverse transform later
```

4.4 — Multicollinearity Check:

```python
# Compute correlation matrix
# Flag feature pairs with |correlation| > 0.85
# Do NOT drop automatically — document and let CatBoost handle it
# (CatBoost is robust to correlated features unlike linear models)
```

4.5 — Temporal Train/Test Split (CRITICAL — NO RANDOM SPLIT):

```python
# Financial data is time-series — random split causes data leakage
# Use WALK-FORWARD split:

# Training set:   2015-01-01 to 2021-12-31 (~7 years)
# Validation set: 2022-01-01 to 2022-12-31 (hyperparameter tuning)
# Test set:       2023-01-01 to 2024-12-31 (final evaluation — touch ONCE)

# NEVER use future data in training features
# Add a 21-day buffer between train and test to prevent leakage from
# rolling feature windows
```

4.6 — Market Regime Labels (for robustness analysis):

```python
```

```
# Label each date with market regime:
# Bull: VIX < 20 and S&P 500 / DAX > 200-day MA
# Crisis: VIX > 30
# Neutral: otherwise
# Save regime labels — used later in Step 8 regime robustness test
```

Save outputs:

- `data/processed/train.csv`
- `data/processed/validation.csv`
- `data/processed/test.csv`
- `models/scaler.pkl`

STEP 5 — BASELINE MODEL WITH SHAP (`src/05_baseline_model.py`)

5.1 — CatBoost Classifier:

```python
from catboost import CatBoostClassifier

# Initial model with default parameters
model = CatBoostClassifier(
    loss_function='MultiClass',   # 3-class: Buy/Hold/Sell
    eval_metric='Accuracy',
    random_seed=42,
    verbose=100
)
model.fit(X_train, y_train, eval_set=(X_val, y_val), early_stopping_rounds=50)
```

5.2 — Hyperparameter Tuning (Grid Search with Time-Series CV):

```python
```

```
# Use TimeSeriesSplit from sklearn — NOT regular KFold
from sklearn.model_selection import TimeSeriesSplit

param_grid = {
    'iterations':        [500, 1000, 2000],
    'learning_rate':     [0.01, 0.05, 0.1],
    'depth':             [4, 6, 8],
    'l2_leaf_reg':       [1, 3, 5],
    'bagging_temperature': [0.5, 1.0]
}

# Run grid search with 5-fold TimeSeriesSplit
# Log all parameter combinations and their validation accuracy
# Select best model
# Save: results/tables/hyperparameter_tuning_results.csv
```

5.3 — SHAP Baseline Explanation:

```python
import shap

explainer = shap.TreeExplainer(best_catboost_model)
shap_values = explainer.shap_values(X_test)

# For multi-class: shap_values is a list of arrays [Buy, Hold, Sell]

# Compute per-feature SHAP importance (mean absolute):
shap_importance = pd.DataFrame({
    'feature': feature_cols,
    'shap_buy':  np.abs(shap_values[0]).mean(axis=0),
    'shap_hold': np.abs(shap_values[1]).mean(axis=0),
    'shap_sell': np.abs(shap_values[2]).mean(axis=0),
    'shap_mean': np.abs(np.array(shap_values)).mean(axis=(0,1))
}).sort_values('shap_mean', ascending=False)

# Save: results/tables/shap_scores.csv
# Plot: results/plots/shap_summary_plot.png (beeswarm plot)
# Plot: results/plots/shap_bar_plot.png (mean importance)
```

5.4 — Confusion Matrix and Classification Report:

```python
```

```python
from sklearn.metrics import confusion_matrix, classification_report

# Confusion matrix on test set
cm = confusion_matrix(y_test, y_pred, labels=['Buy', 'Hold', 'Sell'])

# Plot confusion matrix heatmap
# Save: results/plots/confusion_matrix.png

# Classification report
report = classification_report(y_test, y_pred,
                               target_names=['Buy', 'Hold', 'Sell'],
                               output_dict=True)
# Save: results/tables/classification_report.csv

# Metrics to compute and save:
# - Accuracy (overall)
# - Precision per class
# - Recall per class
# - F1-score per class (weighted and macro)
# - Cohen's Kappa (important for imbalanced multi-class)
# - Matthews Correlation Coefficient
```

STEP 6 — CAUSAL DISCOVERY (`src/06_causal_discovery.py`)

6.1 — DirectLiNGAM (Primary Method):

```python
```

```python
from lingam import DirectLiNGAM

# CRITICAL: Apply prior information (b) from Takahashi et al. (2024):
# Prior (b): investment_decision is the SINK variable —
# it does not cause any other variable

# Prior information matrix:
# prior_knowledge[i][j] = 1  means Xi -> Xj is forbidden
# prior_knowledge[i][j] = -1 means Xi -> Xj is required
# prior_knowledge[i][j] = 0  means no constraint

n_features = len(feature_cols)
prior_knowledge = np.zeros((n_features + 1, n_features + 1))
target_idx = n_features  # Last column = investment_decision
# Forbid edges FROM investment_decision TO any feature:
prior_knowledge[target_idx, :] = 1
prior_knowledge[target_idx, target_idx] = 0  # self-loop not needed

model_lingam = DirectLiNGAM(prior_knowledge=prior_knowledge)
model_lingam.fit(X_train_with_labels)

adj_matrix = model_lingam.adjacency_matrix_
```

6.2 — PC Algorithm (Cross-check / Validation):

```python
from causallearn.search.ConstraintBased.PC import pc
from causallearn.utils.cit import fisherz

# Run PC algorithm independently
cg = pc(X_train_array, alpha=0.05, indep_test=fisherz)
pc_adj_matrix = cg.G.graph

# Compare DirectLiNGAM and PC results:
# Count edges agreed upon by both methods
# Flag edges that disagree — these are uncertain causal claims
# Save agreement analysis: results/tables/causal_method_agreement.csv
```

6.3 — DAG Stability Analysis (Bootstrap):

```python
```

```
# Bootstrap 100 samples of training data (80% each)
# Re-run DirectLiNGAM on each sample
# For each directed edge A -> B, compute:
#   stability_score = (count edge present across 100 runs) / 100
#
# Flag edges with stability_score < 0.5 as UNSTABLE
# Only report edges with stability_score >= 0.7 as reliable
# Save: results/tables/dag_stability_scores.csv
# Plot: results/plots/dag_stability_heatmap.png
```

6.4 — Regime Stability Test:

```python
# Run DirectLiNGAM separately on:
#   - Bull market periods (VIX < 20)
#   - Crisis periods (VIX > 30)
#   - Neutral periods
# Compare adjacency matrices across regimes
# Compute Jaccard similarity between regime-specific DAGs
# Document which causal edges change across market regimes
# Save: results/tables/regime_stability_report.csv
```

6.5 — Plot Causal Graph:

```python
import networkx as nx
import matplotlib.pyplot as plt

# Create directed graph from adjacency matrix
# Use threshold: only show edges with |weight| > 0.1
# Color nodes: blue=macro features, green=stock features, red=target
# Edge width proportional to causal strength
# Save: results/plots/causal_graph_directlingam.png
# Save adjacency matrix: models/adj_matrix_directlingam.npy
```

STEP 7 — LEWIS SCORES (`src/07_lewis_scores.py`)

CRITICAL: This implements the multi-class extension of LEWIS. The original Takahashi paper used binary classification. This thesis extends it to 3-class (Buy/Hold/Sell) — this is an explicit open question from the paper and a key academic contribution.

7.1 — Discretization:

```python
# LEWIS requires discretized features
# Use equal-width discretization with n_bins=10 (same as Takahashi 2024)
from sklearn.preprocessing import KBinsDiscretizer

discretizer = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='uniform')
X_train_disc = discretizer.fit_transform(X_train)
X_test_disc = discretizer.transform(X_test)
```

7.2 — Multi-class LEWIS (Pairwise):

```python
# For 3-class extension: compute LEWIS scores pairwise
# Buy vs Rest: treat Buy=positive, Hold+Sell=negative
# Sell vs Rest: treat Sell=positive, Buy+Hold=negative
#
# For each pairwise comparison and each feature Xi, compute:

# Necessity (Nec): P(outcome changes | decrease Xi, currently positive class)
# Nec(x, x') = [P(o'|do(X=x')) - P(o'|x)] / P(o|x)

# Sufficiency (Suf): P(outcome changes | increase Xi, currently negative class)
# Suf(x, x') = [P(o|do(X=x)) - P(o|x')] / P(o'|x')

# Necessity+Sufficiency (Nesuf):
# Nesuf(x, x') = P(o'|do(X=x')) - P(o'|do(X=x))

# Use the causal graph (adj_matrix from Step 6) to compute do-calculus queries
# If causal graph is available: use backdoor adjustment
# If feature is independent of target: Nesuf = 0 (from paper)

# Global score per feature: maxNesuf(Xi) = max over all (x,x') pairs

# Compute for:
# (a) WITH causal graph (causal LEWIS)
# (b) WITHOUT causal graph — assume P(O|do(X)) = P(O|X) (no-graph baseline)
# This replicates the comparison in Takahashi 2024 Table V-X
```

7.3 — Reversal Probabilities:

```python
```

```
# For each feature, compute:
# Nec score: probability that reducing this feature flips a Buy to non-Buy
# Suf score: probability that increasing this feature flips a Sell to Buy
# Plot reversal probabilities as bar chart (replicate Figure 7 of paper)
# Save: results/plots/reversal_probabilities.png
# Save: results/tables/reversal_scores.csv
```

7.4 — Save all LEWIS outputs:

```python
# results/tables/lewis_scores_causal.csv     (with DAG)
# results/tables/lewis_scores_no_graph.csv   (without DAG)
# results/tables/lewis_vs_shap_comparison.csv
```

STEP 8 — EVALUATION FRAMEWORK (`src/08_evaluation.py`)

This is the most important section for thesis validity. Run ALL of these.

8.1 — Standard ML Metrics:

```python
# Already computed in Step 5 but re-run on final test set here
# Accuracy, Precision, Recall, F1 (per class + weighted/macro)
# Cohen's Kappa, Matthews Correlation Coefficient
# AUC-ROC (one-vs-rest for multi-class)
# AUC-PR (precision-recall curve per class)
# Calibration curve (reliability diagram) — important for investment decisions
```

8.2 — Counterfactual Validity Metrics:

```python
```

```
# For each explanation, evaluate:
# VALIDITY:   Does the suggested counterfactual actually flip the prediction?
# SPARSITY:   How many features need to change? (fewer = better)
# PROXIMITY:  How close is the counterfactual to the original instance?
# PLAUSIBILITY: Is the counterfactual within the training data distribution?
# ACTIONABILITY: Can the suggested change actually be made by a decision-maker?

# Actionability constraints (CRITICAL — this was a flaw in original code):
# NOT actionable: VIX, ECB rate, EUR/USD (macro — cannot be changed by investor)
# ACTIONABLE: momentum, volume, sector rotation
# Flag non-actionable counterfactuals as INFEASIBLE
# Compute: % feasible counterfactuals per class
# Save: results/tables/counterfactual_validity_report.csv
```

8.3 — DAG Evaluation (Structural):

```python
# Evaluate causal graph quality using:
# Spearman Rank Correlation (SPR) between estimated and proxy-true Nesuf order
# Mean Absolute Error (MAE) of Nesuf scores: causal vs no-graph
# (Replicate Table V-X from Takahashi 2024 for investment context)
# Save: results/tables/dag_evaluation_metrics.csv
```

8.4 — Regime Robustness Test:

```python
# Evaluate model performance SEPARATELY for each market regime:
# - Bull market accuracy, F1
# - Crisis period accuracy, F1
# - Neutral period accuracy, F1
# If performance drops significantly in crisis periods: document as limitation
# Save: results/tables/regime_robustness_report.csv
# Plot: results/plots/accuracy_by_regime.png
```

8.5 — Temporal Stability (Rolling Window):

```python
# Evaluate model performance in rolling 6-month windows on test set
# Plot accuracy over time to detect model degradation
# Save: results/plots/rolling_accuracy.png
```

8.6 — Sector-Level Analysis:

```python
```

```
# Breakdown of prediction accuracy per GICS sector
# (Technology, Financials, Healthcare, Energy, etc.)
# Identify which sectors CatBoost predicts best/worst
# Save: results/tables/sector_accuracy_report.csv
# Plot: results/plots/accuracy_by_sector.png
```

STEP 9 — SHAP VS LEWIS COMPARISON (`src/09_comparison.py`)

This is the core academic contribution. Produce the full comparison.

9.1 — Side-by-side Feature Ranking:

```python
# For each feature, show:
# - SHAP importance (correlation-based)
# - LEWIS maxNesuf with causal graph
# - LEWIS maxNesuf without causal graph
# - Rank from each method
# - Rank agreement score (Spearman between SHAP rank and LEWIS rank)

comparison_df = pd.DataFrame({
    'feature':          feature_cols,
    'shap_importance':  shap_importance_scores,
    'lewis_causal':     lewis_causal_scores,
    'lewis_no_graph':   lewis_no_graph_scores,
    'rank_shap':        rankdata(-shap_importance_scores),
    'rank_lewis':       rankdata(-lewis_causal_scores),
})

spearman_corr, p_value = spearmanr(comparison_df['rank_shap'],
                                   comparison_df['rank_lewis'])
# Save: results/tables/shap_vs_lewis_comparison.csv
# Plot: results/plots/shap_vs_lewis_comparison.png
```

9.2 — Identify Key Disagreements:

```python
# Find features where SHAP and LEWIS strongly disagree in ranking
# These are potential spurious correlations that SHAP is capturing
# Document each disagreement with domain justification
# Example: "VIX has high SHAP importance (correlated with crisis) but
#           low LEWIS Nesuf (cannot be intervened upon)"
```

```python
# Load survey results from SQLite/MongoDB
# Compute: % respondents who preferred causal explanation vs SHAP
# Compute: mean trust rating per explanation type
# Statistical test: paired t-test or Wilcoxon signed-rank test
# Save: results/tables/expert_survey_analysis.csv
# NOTE: Survey sample must have >=30 responses to be statistically meaningful
#       Document this as a limitation if fewer responses are collected
```

STEP 10 — GOVERNANCE FRAMEWORK (`src/10_governance.py`)

10.1 — Generate Regulatory Compliance Report:

Produce a structured text/JSON output that maps each pipeline component to regulatory requirements. This is NOT just a checklist — each item must state WHAT the pipeline does and WHY it satisfies the requirement.

```python
```

```python
compliance_report = {
    "EU_AI_Act_Article_13": {
        "requirement": "Transparency — High-risk AI systems must provide
                        information enabling human oversight",
        "implementation": "LEWIS counterfactual explanations provide
                           feature-level causal necessity and sufficiency
                           scores, enabling investment managers to understand
                           the causal drivers of each recommendation",
        "evidence": "results/tables/lewis_scores_causal.csv",
        "gap": "Multi-class extension of LEWIS not yet peer-reviewed"
    },
    "EU_AI_Act_Article_14": {
        "requirement": "Human oversight — meaningful ability to override",
        "implementation": "Streamlit app presents causal explanation alongside
                           prediction, allowing analyst to approve/reject",
        "evidence": "app.py survey functionality"
    },
    "BaFin_AI_Guidelines_2024": {
        "requirement": "Auditability of AI decision processes in banking",
        "implementation": "All model artifacts, DAG, scores, and explanations
                           are saved with timestamps and reproducible pipeline",
        "evidence": "models/ directory, pipeline.py"
    },
    "MiFID_II_Article_25": {
        "requirement": "Suitability — investment recommendations must be
                        based on clients risk profile and justified",
        "implementation": "Counterfactual feasibility constraints ensure
                           recommendations are based on actionable,
                           non-macro features only",
        "evidence": "results/tables/counterfactual_validity_report.csv"
    }
}

# Save: results/reports/regulatory_compliance_report.json
# Save: results/reports/regulatory_compliance_report.md (human-readable)
```

MASTER PIPELINE RUNNER (`src/pipeline.py`)

```python
```

```python
"""
Master pipeline runner. Executes all steps in order.
Usage: python src/pipeline.py --steps all
       python src/pipeline.py --steps 1,2,3
       python src/pipeline.py --steps 5,6,7  (skip download if data exists)
"""

import argparse
import logging
from pathlib import Path

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.FileHandler('results/pipeline.log'),
        logging.StreamHandler()
    ]
)

STEPS = {
    1: ('Data Download',        '01_data_download'),
    2: ('Feature Engineering',  '02_feature_engineering'),
    3: ('Labeling',             '03_labeling'),
    4: ('Data Preparation',     '04_data_preparation'),
    5: ('Baseline + SHAP',      '05_baseline_model'),
    6: ('Causal Discovery',     '06_causal_discovery'),
    7: ('LEWIS Scores',         '07_lewis_scores'),
    8: ('Evaluation',           '08_evaluation'),
    9: ('Comparison',           '09_comparison'),
    10: ('Governance Report',   '10_governance'),
}

# Run each step, catch and log errors, continue to next step
```

REQUIREMENTS FILE (`requirements.txt`)

```
yfinance>=0.2.36
pandas>=2.0.0
numpy>=1.24.0
scikit-learn>=1.3.0
catboost>=1.2.0
shap>=0.44.0
lingam>=1.8.3
```

```
causal-learn>=0.1.3.8
networkx>=3.0
matplotlib>=3.7.0
seaborn>=0.12.0
scipy>=1.10.0
statsmodels>=0.14.0
pandas-datareader>=0.10.0
streamlit>=1.28.0
pymongo>=4.0.0
tqdm>=4.65.0
joblib>=1.3.0
fredapi>=0.5.0
```

WHAT MAKES THIS THESIS-GRADE — CHECKLIST

Verify ALL of these before submission:

Data Quality:

- [ ] Real data only — zero synthetic rows
- [ ] Stationarity checked and documented per feature (ADF test)
- [ ] No data leakage (temporal split with buffer)
- [ ] Class balance documented (not artificially resampled)
- [ ] Missing data handling documented with row counts

Model Quality:

- [ ] Hyperparameter tuning with TimeSeriesSplit (not random KFold)
- [ ] Confusion matrix produced for all 3 classes
- [ ] Classification report: precision, recall, F1 per class
- [ ] Cohen's Kappa computed (handles class imbalance)
- [ ] Calibration curve produced

Causal Discovery Quality:

- [ ] DirectLiNGAM with prior information (b) — sink variable constraint
- [ ] PC algorithm cross-check — edge agreement rate documented
- [ ] Bootstrap stability analysis (100 runs) — unstable edges flagged
- [ ] Regime stability test (Bull / Crisis / Neutral) — regime shift documented
- [ ] DAG evaluation: SPR and MAE (replicating Table V-X of paper)

LEWIS Quality:

- [ ] Multi-class extension documented as novel contribution
- [ ] Scores computed WITH and WITHOUT causal graph

- [ ] Counterfactual feasibility constraints applied (macro features flagged)
- [ ] Validity, sparsity, proximity, plausibility metrics computed

Comparison Quality:

- [ ] SHAP vs LEWIS rank correlation (Spearman) computed
- [ ] Key disagreements identified and explained with domain knowledge
- [ ] Expert survey collected with statistical test

Regulatory Quality:

- [ ] EU AI Act Article 13 and 14 mapped with evidence
- [ ] BaFin guidelines mapped with evidence
- [ ] MiFID II mapped with evidence
- [ ] Gaps honestly documented

KNOWN LIMITATIONS TO DOCUMENT IN THESIS

These are NOT failures — documenting them is academic integrity:

1. DirectLiNGAM assumes linear non-Gaussian relationships — financial data may violate this in nonlinear regimes
2. DAG may be unstable during structural breaks (COVID-19, 2022 rate hikes) — document which periods show instability
3. Multi-class LEWIS extension is novel — no peer-reviewed validation exists yet (this IS your contribution)
4. Counterfactuals are mathematically minimal, not economically optimal — feasibility constraints partially address this but not completely
5. Expert survey may have small sample size — document minimum required for statistical significance
6. S&P 500 companies are US-listed but model is used in EU regulatory context — justify why global validation strengthens generalizability

---

CODING STANDARDS FOR READABILITY

Every file must follow these rules so the code is readable for thesis submission:

```python
```

```python
# At top of every file:
"""
Module: [step name]
Purpose: [one sentence]
Inputs:  [list of input files/data]
Outputs: [list of output files]
Reference: Takahashi et al. (2024) arXiv:2402.02678
"""

# Every function must have:
def compute_lewis_scores(X, y, adj_matrix, feature_cols, n_bins=10):
    """
    Compute LEWIS Nec/Suf/Nesuf scores for each feature.

    Args:
        X:           Feature matrix (discretized)
        y:           Target labels
        adj_matrix:  Causal adjacency matrix from DirectLiNGAM
        feature_cols: List of feature names
        n_bins:      Number of discretization bins (default=10, per paper)

    Returns:
        DataFrame with columns: feature, Nec, Suf, Nesuf, maxNesuf

    Reference:
        Takahashi et al. (2024), Section II-B, Equations (3)-(8)
    """
```

FINAL OUTPUT ARTIFACTS FOR THESIS

After pipeline.py completes, these files must exist and be referenced in thesis:

File	Thesis Section
results/tables/stationarity_report.csv	Data Preparation
results/tables/label_distribution.csv	Data Section
results/tables/hyperparameter_tuning_results.csv	Model Section
results/plots/confusion_matrix.png	Results Section
results/tables/classification_report.csv	Results Section
results/plots/shap_summary_plot.png	Baseline XAI Section
results/plots/causal_graph_directlingam.png	Causal Discovery Section
results/tables/dag_stability_scores.csv	Causal Validation Section
results/tables/causal_method_agreement.csv	Causal Validation Section
results/tables/lewis_scores_causal.csv	LEWIS Section
results/plots/reversal_probabilities.png	LEWIS Section
results/plots/shap_vs_lewis_comparison.png	Comparison Section
results/tables/counterfactual_validity_report.csv	Evaluation Section
results/tables/regime_robustness_report.csv	Robustness Section
results/tables/expert_survey_analysis.csv	User Study Section
results/reports/regulatory_compliance_report.md	Governance Section

Build prompt prepared for Frankfurt School Master Thesis — Causal XAI for Investment AI
Reference: Takahashi, Shimizu, Tanaka (2024), arXiv:2402.02678